

SIMATS ENGINEERING

ASSESSMENT TOOL 2 — CASE STUDY

Computer Networks

Course Name	Computer Networks
Course Code	CSA07
Course Outcome (CO3)	Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions
Assessment Tool Weightage	40%
Total Questions	4
Total Marks	40

Case Study 1: Video Conferencing Performance Degradation

A multinational IT company's cloud-based video conferencing platform suffers voice distortion, frozen video frames, and delayed communication during peak hours. Monitoring shows 8% packet loss, 180 ms jitter, and fluctuating bandwidth. The administrator suspects the transport protocol or application-level buffering strategy.

1. Is the problem caused by the transport protocol (TCP/UDP) or the application layer?

The symptoms described — voice distortion, frozen video frames and delayed communication — point primarily to the transport layer, but the way the application layer reacts to transport-layer behaviour amplifies the problem. The two clues that make this a transport-layer issue first are the 8% packet loss and 180 ms jitter. If the platform is running over TCP, the protocol's reliability mechanism forces retransmission of every lost segment before any later data can be delivered to the application (head-of-line blocking). With 8% loss, a large fraction of segments must be resent, and each retransmission waits for a retransmission timeout or triple-duplicate-ACK, which is exactly what produces frozen frames and delayed audio. TCP's congestion control (slow start / AIMD) would also cut the sending rate sharply on repeated loss, explaining the fluctuating bandwidth utilisation.

If, instead, the platform is running over UDP (the normal choice for real-time media), the loss and jitter are not "fixed" by the transport layer at all — UDP simply delivers what arrives, with no retransmission and no ordering guarantee. In that case the voice distortion and frozen frames are an application-layer symptom: the codec and jitter buffer are not compensating adequately for 180 ms of jitter and 8% loss. So the diagnosis has two layers: the root cause of the packet loss and jitter itself is a network/transport-level condition (congested links, queuing delay, insufficient bandwidth during peak hours), while the visible degradation the users experience is shaped by how the transport protocol and the application-level buffering/error-concealment strategy respond to that condition. In short: the underlying trigger is transport/network-level congestion, but the application layer's buffering strategy determines whether that congestion becomes a minor glitch or a frozen call.

2. Justification based on real-time multimedia communication requirements

Real-time multimedia traffic (voice and video) has fundamentally different requirements from bulk data transfer, and this is why the diagnosis above holds:

- Timeliness over completeness: A voice sample or video frame that arrives late is useless — it cannot be "caught up" on, unlike a file where a late byte is still valid. TCP's insistence on

reliable, in-order delivery therefore directly conflicts with real-time needs: retransmitting a lost packet after its playback deadline has passed only adds delay without adding value.

- Tolerance for loss, intolerance for delay: The human ear and eye can tolerate small amounts of packet loss (a brief click or a skipped frame) far better than they tolerate variable delay (jitter) or large one-way latency, which is why ITU-T recommends keeping one-way delay under about 150 ms for good voice quality.
- Jitter directly degrades perceived quality: 180 ms of jitter is very high compared to the typical 20-50 ms tolerated by a well-tuned jitter buffer; when jitter exceeds the buffer's compensation range, frames arrive too late to be played and are perceived as freezing.
- Bandwidth fluctuation hurts adaptive codecs: Modern conferencing codecs (Opus, VP8/VP9, H.264 with SVC) adapt bitrate to available bandwidth; sharp fluctuations force frequent re-negotiation of quality, which appears to users as blurring or stalling.

These requirements are precisely why UDP (often wrapped in RTP/RTCP for sequencing and quality feedback) is the standard transport for conferencing platforms — it deliberately sacrifices reliability to preserve timeliness, and pushes the responsibility for handling loss and jitter up to the application layer, where it can be handled in ways that are aware of the real-time deadline (e.g., concealment) rather than blindly retransmitting.

3. Recommended protocol-level and application-level improvements

Protocol-level improvements: Ensure the media path uses UDP-based RTP/RTCP rather than TCP for audio/video streams (TCP may still be used for signalling/control, e.g., SIP or WebSocket). Enable RTCP feedback (NACK, RTCP-XR) so the sender can detect loss quickly and retransmit only time-critical, small amounts of data selectively. Use ECN (Explicit Congestion Notification) and a congestion-controlled RTP profile (e.g., Google Congestion Control / WebRTC's GCC, or L4S where supported) so the sender adapts bitrate before loss becomes severe, rather than after.

Application-level improvements: Implement an adaptive jitter buffer that dynamically resizes based on measured jitter (instead of a fixed buffer), so 180 ms of jitter is absorbed smoothly rather than causing frame drops. Add Forward Error Correction (FEC) and Packet Loss Concealment (PLC) so the codec can reconstruct or smoothly interpolate over the 8% lost packets instead of freezing. Use scalable video coding (SVC) or simulcast so video quality degrades gracefully (lower resolution/frame rate) instead of freezing.

when bandwidth drops. Prioritise audio over video when bandwidth is constrained, since audio continuity matters more to comprehension than video smoothness.

Network-level supporting measures: Apply QoS/DSCP marking to prioritise real-time media traffic on the corporate network, provision additional peak-hour bandwidth or use a dedicated media relay (TURN/SFU) closer to users to reduce queuing delay, and monitor per-link jitter/loss so congested peak-hour segments can be identified and traffic-engineered.

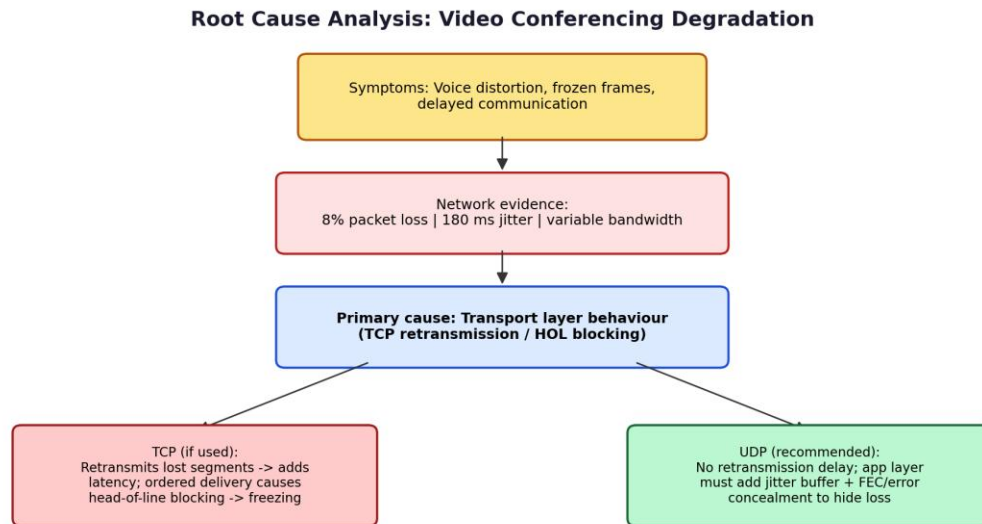


Figure 1: Root-cause chain from observed symptoms to transport-layer cause and the recommended UDP-based remedy.

Case Study 2: High DNS Lookup Time for a Global Web Application

A multinational software company hosts web services across continents. Employees report long homepage load delays; average DNS lookup time is about 800 ms. The organisation wants to redesign its DNS infrastructure for lower latency while keeping service available during server failures.

1. Reasons behind the high DNS resolution time (~800 ms)

- Centralised / single-region authoritative server: If all authoritative DNS records live on one server in one region, every query from a distant continent must cross long-haul, high round-trip-time (RTT) links, adding tens to hundreds of milliseconds per hop.
- Full iterative/recursive resolution on cache miss: When the local resolver's cache is cold (no cached entry), it must walk the hierarchy — root server -> TLD server -> authoritative server — each hop adding its own RTT, which compounds to the observed ~800 ms.

- Low or absent caching / aggressive TTLs: If TTL values are set very low (to allow fast failover) or caching is disabled for reliability reasons, resolvers cannot reuse previous answers and must repeat the full lookup very often.
- No pre-fetching: Without proactive refresh of soon-to-expire records, users are far more likely to hit a cold cache exactly when TTL expires, so the full resolution delay is experienced live by the user rather than being hidden in the background.
- No Anycast routing: A unicast DNS server presents one fixed network location, so every user, regardless of geography, is routed to the same physical server rather than the nearest instance.
- DNS server overload during peak/promotional traffic: A single primary server handling global query volume can queue requests, adding processing delay on top of network delay.

2. Proposed DNS architecture (Anycast DNS, pre-fetching, TTL optimisation) to reach < 50 ms

Anycast DNS: Announce the same authoritative DNS IP address from multiple geographically distributed Points of Presence (PoPs) using BGP anycast. Internet routing then automatically delivers each user's query to the topologically nearest PoP, cutting the physical distance (and therefore RTT) a query must travel — this alone can take resolution from several hundred milliseconds down to tens of milliseconds for most users.

DNS pre-fetching: Configure recursive resolvers (and, where possible, the browser/OS stub resolver) to proactively re-resolve frequently accessed hostnames slightly before their TTL expires, and to prefetch DNS for links likely to be clicked next. This ensures that, from the user's perspective, a fresh answer is almost always already cached when needed, hiding resolution latency behind normal browsing activity rather than exposing it on the critical path.

TTL optimisation: Set a TTL that balances cache-hit ratio against failover speed — typically in the 60-300 second range for records that rarely change, rather than very low (a few seconds) TTLs that force constant re-resolution, or very high TTLs that slow failover. A tiered approach works well: longer TTLs for stable records (e.g., the main domain) and shorter TTLs only on records that participate in failover/load-balancing decisions.

Supporting layers: Deploy a hierarchy of caching recursive resolvers close to each office/region, use multiple authoritative name-server providers (multi-vendor / secondary DNS) for redundancy, and enable DNSSEC validation caching so security checks do not add repeated latency.

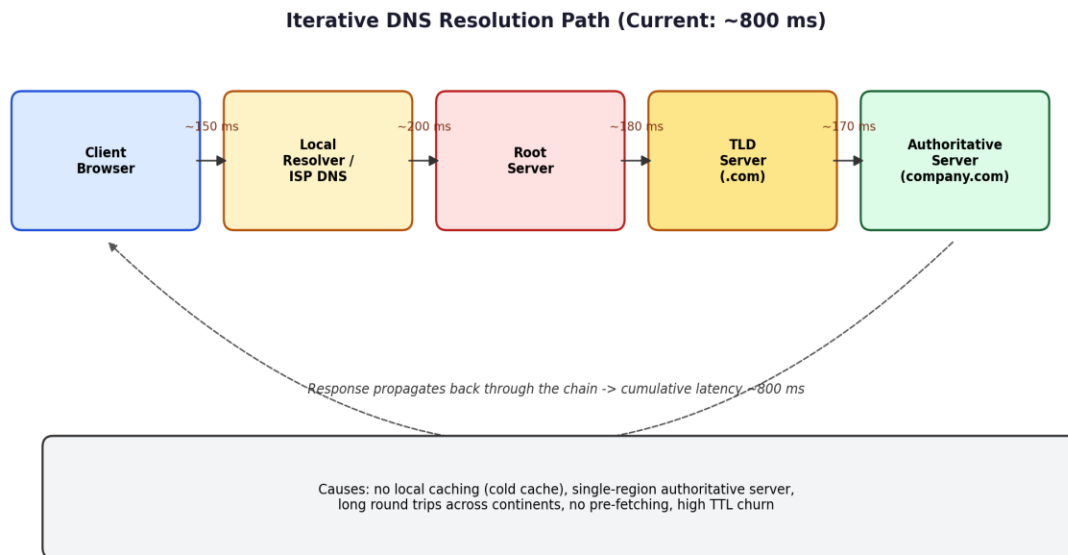


Figure 2: Current iterative resolution path — each hop across the hierarchy adds latency, totalling roughly 800 ms.

3. Advantages and limitations of the proposed architecture

Dimension	Advantages	Limitations
Performance	Queries served by the nearest PoP; typical lookups fall to well under 50 ms; pre-fetch hides remaining latency	Anycast performance depends on ISP BGP routing quality — a poorly peered region may still see higher RTT
Scalability	New PoPs can be added without changing the published IP; load is naturally spread across PoPs by routing	Requires BGP anycast expertise and coordination with multiple ISPs / cloud DNS providers to deploy correctly
Availability	If one PoP fails, BGP reconvergence automatically routes users to the next-nearest healthy PoP	BGP convergence is not instantaneous (seconds), so a very short availability gap can occur during a PoP failure
Caching efficiency	Optimised TTLs plus pre-fetching keep cache-hit ratios high, minimising full resolutions	Longer TTLs slightly delay how fast clients pick up legitimate record changes (e.g., planned IP migration)
Operational cost	Reduces load on any single server; improves user experience globally	Running multiple geographically distributed, BGP-capable DNS PoPs costs more than a single server deployment

Case Study 3: Centralised DNS Bottleneck for a Global Cloud Provider

A cloud provider serving millions of users worldwide sees intermittent access delays during promotional events. DNS queries take nearly 800 ms because all requests go to a single primary DNS server. The company wants a highly available DNS solution for global traffic.

4. Impact of centralised DNS architecture on application performance

A single, centralised DNS server creates several compounding performance problems, especially under the traffic spikes typical of promotional events. First, it introduces a geographic latency penalty: every user worldwide, regardless of their own location, must send their query across the network to the one server's location, so users far from that location experience high RTT purely from physical distance and the number of transit networks the query crosses.

Second, it creates a query-volume bottleneck. Because there is only one server (or one small cluster in one location) to answer all global queries, a promotional event that suddenly increases traffic multiplies the queueing delay at that server — queries pile up waiting to be processed, so response times degrade further for everyone simultaneously, which matches the "intermittent delays" observed. Third, it is a single point of failure: if that server, its host, or the network path to it experiences any disruption, DNS resolution fails for the entire user base at once, taking down access to every cloud-hosted application that depends on it — even though the applications themselves may be perfectly healthy. Finally, centralised DNS undermines any downstream load-balancing or failover strategy the application layer might have, because if users cannot resolve the name quickly (or at all), it does not matter how well-distributed the backend application servers are.

5. Design of a distributed DNS solution minimising latency while maintaining high availability

The proposed design replaces the single primary server with a globally distributed, redundant DNS service built on four coordinated layers:

- **Anycast edge layer:** The same authoritative IP address is announced from PoPs on multiple continents (North America, Europe, Asia, South America, Africa, Oceania). BGP automatically steers each user's query to the nearest healthy PoP, eliminating the long-haul latency penalty and, because multiple PoPs share the load, also eliminating the single query-volume bottleneck.
- **Redundant authoritative back end:** Each PoP is backed by at least two independent authoritative name-server clusters (ideally from two different DNS providers/vendors) so that a failure of one provider's infrastructure does not take down resolution entirely — this removes the single point of failure identified above.

- **Distributed recursive caching layer:** Regional recursive resolvers (deployed by the company or via a managed DNS/CDN provider) cache answers close to users, absorbing repeat queries so most requests never need to reach the authoritative layer at all.
- **Health-checked failover:** Authoritative records are paired with active health checks on the underlying application endpoints, so DNS answers automatically stop pointing to unhealthy backend regions, keeping resolution useful even during partial outages.

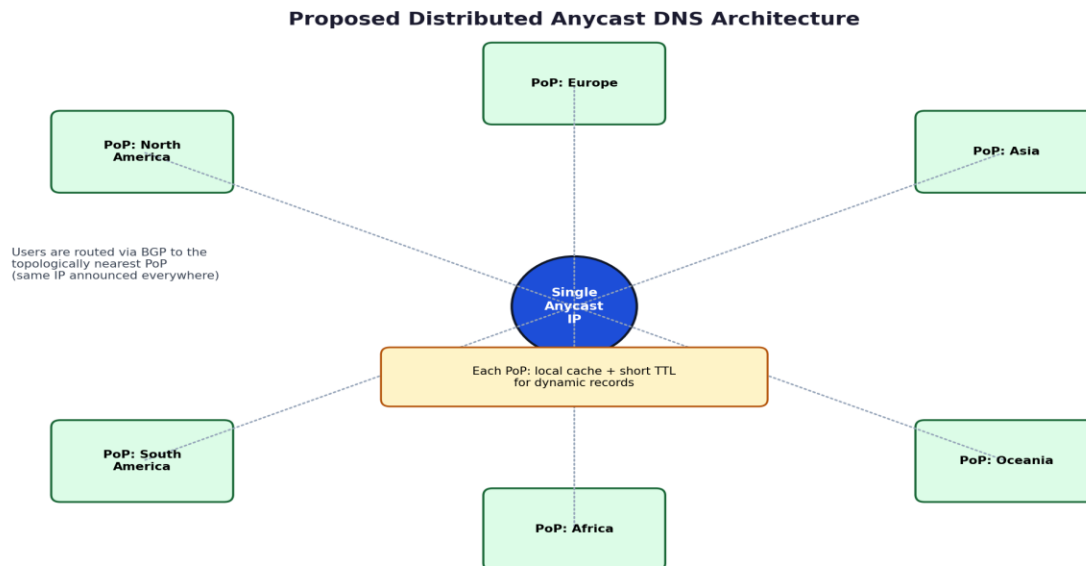


Figure 3: Distributed Anycast DNS architecture — a single announced IP served from six regional PoPs, each with local caching.

6. How TTL, Anycast routing, and DNS caching influence performance and fault tolerance

TTL values: TTL directly trades off performance against agility. A longer TTL (minutes) means resolvers reuse cached answers far more often, sharply cutting the average lookup time and load on authoritative servers — but it also means that if a server fails, some resolvers keep using the stale (dead) answer until the TTL expires. Shorter TTLs improve fault-tolerance responsiveness (faster failover) at the cost of more frequent full lookups. The practical answer is tiered TTLs: longer for stable records, shorter only on records tied to active failover.

Anycast routing: Anycast improves both performance and fault tolerance simultaneously. Performance improves because queries are answered by the nearest PoP rather than crossing the globe. Fault tolerance improves because if one PoP fails or is disconnected, BGP simply stops advertising that route and traffic converges onto the next-nearest healthy PoP automatically, without any client-side reconfiguration or manual DNS change.

DNS caching: Caching at every layer (browser, OS stub resolver, ISP recursive resolver, and company-operated recursive resolvers) means the vast majority of queries are answered locally in microseconds rather than requiring a network round trip at all. This dramatically reduces average load on the authoritative layer, which in turn means the authoritative servers have more headroom to absorb traffic spikes (such as a promotional event) without queuing delay, indirectly improving both performance and resilience.

Together, tiered TTLs, Anycast routing, and layered caching form a mutually reinforcing design: caching minimises how often the network is used at all, Anycast minimises the latency and blast radius when the network is used, and well-chosen TTLs balance how quickly the system can recover from failure against how much repeated lookup overhead is acceptable.

Case Study 4: TCP Latency Spike on an Electronic Stock Trading Platform

A financial institution's electronic trading platform handles thousands of orders per second. During market opening hours, average order acknowledgment latency rises from 1 ms to 40 ms. Engineers observe increased TCP retransmissions, longer connection establishment times, and excessive buffering.

1. Three TCP-related factors responsible for the latency increase

a) Flow control (receive window exhaustion): At market open, the huge burst of incoming order and acknowledgment traffic can fill a receiver's socket buffer faster than the receiving application (matching engine or client) can read from it. TCP's flow-control mechanism then shrinks the advertised receive window, and once it reaches zero the sender must pause entirely and wait for a window-update ACK before sending more data — directly adding stall time to every affected order message.

b) Nagle's algorithm: Nagle's algorithm delays sending small TCP segments, batching them until an outstanding segment is acknowledged, in order to improve network efficiency for bulk transfers. Trading messages (orders, acknowledgments) are typically small, so if Nagle's algorithm is enabled, each small order message can sit in the send buffer waiting either for enough data to fill a segment or for the ACK of the previous segment — this waiting is exactly the kind of added delay that turns a 1 ms acknowledgment into tens of milliseconds.

c) SYN queue (accept/backlog queue) limitations: Market open causes a burst of near-simultaneous new TCP connection attempts as client applications reconnect or new sessions are established. If the server's SYN backlog queue is sized too small for this burst, incoming SYNs are dropped once the queue

is full, forcing clients to retransmit their SYN after a timeout — this shows up as "longer connection establishment times" and, indirectly, as bursts of retransmissions right at the point of highest trading activity.

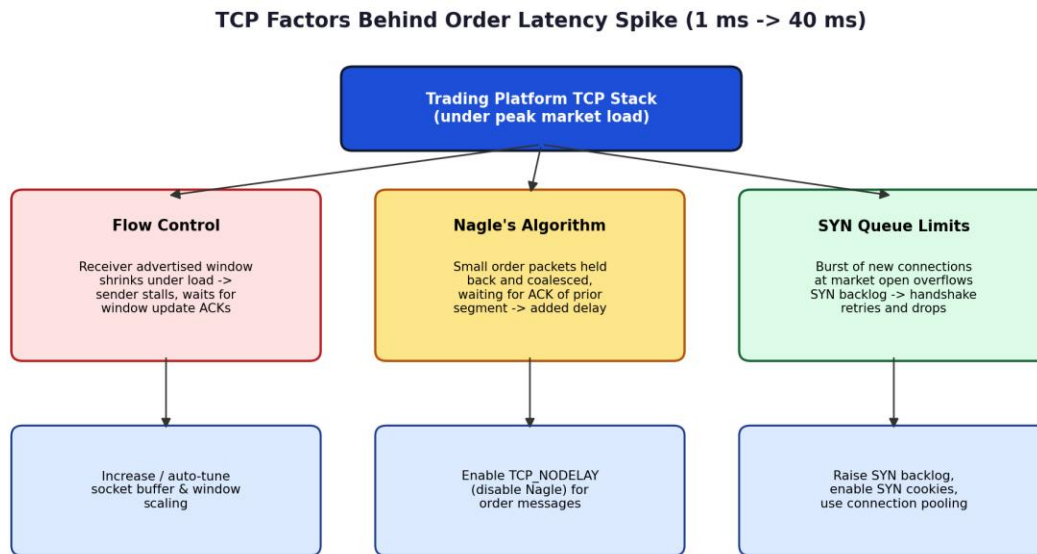


Figure 4: The three TCP factors behind the latency spike, mapped to their effect and the corresponding fix.

2. How each factor affects transaction processing in high-frequency systems

In a high-frequency trading (HFT) context, every millisecond of added latency between an order being sent and its acknowledgment being received translates directly into stale pricing risk, missed fill opportunities, and potential regulatory/best-execution concerns, so these three factors are especially damaging:

- **Flow control stalls:** A window-limited pause means the sender cannot transmit the next order or cancellation instantly; in a market moving quickly at the open, even a few milliseconds of enforced silence can mean an order executes at a worse price than intended, or a cancel arrives too late to prevent an unwanted fill.
- **Nagle's algorithm delay:** Because HFT messages are small and latency-critical (unlike bulk transfers, where Nagle's batching genuinely helps), the algorithm's default behaviour actively works against the platform's goals — it optimises for throughput/efficiency at the direct expense of the per-message latency that HFT strategies depend on.
- **SYN queue drops:** A dropped/retried connection at market open is one of the worst possible failure moments — it is exactly when trading activity (and opportunity) is highest, so any client

unable to establish or re-establish its session in time misses the most volatile, often most profitable, part of the trading session.

Cumulatively, these factors explain both symptoms engineers observed: increased retransmissions (from dropped SYNs and from segments lost under buffer pressure) and excessive buffering (from flow-control windows filling and from Nagle-related coalescing), together producing the jump from 1 ms to 40 ms average acknowledgment latency.

3. Recommended TCP configuration changes to reduce latency and improve reliability

- Disable Nagle's algorithm on trading sockets by setting `TCP_NODELAY`, so small order/acknowledgment messages are sent immediately rather than waiting for batching or a prior ACK.
- Increase and auto-tune socket buffer sizes and enable TCP window scaling, so the advertised receive window can grow large enough to absorb opening-bell bursts without collapsing to zero.
- Ensure the receiving application reads from the socket promptly (e.g., dedicated I/O threads, avoiding blocking processing on the read path) so the receive buffer is drained quickly and the flow-control window does not shrink unnecessarily.
- Increase the SYN backlog queue size (e.g., Linux `net.core.somaxconn` and `net.ipv4.tcp_max_syn_backlog`) and enable SYN cookies, so opening-bell connection bursts are accepted rather than dropped and retried.
- Use persistent, pre-warmed connections / connection pooling for known trading clients instead of establishing a fresh TCP handshake at market open, avoiding the SYN-queue pressure altogether for the highest-priority sessions.
- Consider `TCP_QUICKACK` and tuned delayed-ACK settings so acknowledgments are not artificially delayed on the receiving side, and apply QoS/priority queuing on the network path for trading traffic ahead of less latency-sensitive flows.
- Monitor and alert on retransmission rate and SYN-queue drop counters in real time so configuration limits can be raised proactively before the next high-volume open, rather than reactively after latency has already spiked.